

APTC106:

TALLER DE DESARROLLO WEB Y MOVIL

CASO

FOODPLEASE

Prototipo de Aplicación Móvil Híbrida

Integración Móvil/Web con Despliegue Cloud

Sumativa 4 — Semanas 10 y 11

Alumnos: PRISCILA ARGANARAZ
CARLOS GONZALEZ VILLEGAS
DANIEL HUERTA SALAZAR
OSVALDO MONCADA PERALTA

Profesor: **ERNESTO VIVANCO**

Fecha: **Mayo 19 de 2026**

INDICE

1. INTRODUCCIÓN.....	4
1.1 Motivación	4
1.2 Objetivos del proyecto	4
1.3 Objetivos del informe.....	5
1.4 Estructura del documento.....	5
2. PLANTEAMIENTO DEL AVANCE HACIA LA APLICACIÓN MÓVIL	6
2.1 Contexto: continuidad del proyecto FoodPlease	6
2.2 Brecha que cierra esta entrega	6
2.3 Énfasis declarados por la cátedra	7
3. Análisis crítico: decisiones técnicas del prototipo móvil.....	7
3.1 Criterios de evaluación aplicados	7
3.2 Alternativas evaluadas	8
3.3 Racionalización de descartes	8
3.4 Decisión final: Ionic + Angular + Capacitor	9
4. Arquitectura full-stack integrada.....	10
4.1 Visión general	10
4.2 Capa Backend: Flask + API REST	11
4.3 Capa de comunicación: API REST + CORS	12
4.4 Capa Frontend móvil: Ionic + Angular	13
5. Construcción del prototipo móvil Ionic + Angular	14
5.1 Plantilla utilizada: tabs	14
5.2 Estructura del proyecto.....	14
5.3 Diseño de interfaz	14
5.4 Fluidez de navegación.....	15
5.5 Patrón predominante aplicado: Angular MVC + NgModules + Servicios.....	15
6. Despliegue Cloud del aplicativo web/móvil	16
6.1 Backend desplegado en Render Cloud	16
6.2 Frontend móvil desplegado en Netlify.....	17
6.3 Configuración de producción del frontend	17
6.4 Resumen de URLs públicas de la entrega.....	17
7. Funcionamiento Mínimo Viable	18
7.1 Tabla de estado por módulo del prototipo móvil.....	18
7.2 Tabla de estado del backend Cloud	18

7.3 Coincidencia entre documentación y comportamiento real	19
7.4 Flujo end-to-end reproducible	19
7.5 Reproducibilidad del código en máquina local	19
8. Integración de la propuesta de mejora de la Semana 10	20
8.1 Propuesta de mejora elaborada en la Semana 10	20
8.2 Materialización en la Semana 11	20
8.3 Mejoras adicionales incorporadas durante la implementación	21
9. Conclusiones	22
9.1 Resultados alcanzados	22
9.2 Capacidad crítica: qué salió bien y qué quedó débil.....	22
9.3 Logro de objetivos.....	22
9.4 Marco de acción futura.....	23
10. Glosario y referencias bibliográficas	24
10.1 Glosario	24
10.2 Referencias bibliográficas.....	25
Anexo A — Evidencia del despliegue Cloud y prototipo móvil	26
A.1 Prototipo móvil en ejecución local (validación inicial)	26
A.2 Backend Flask local con logs de la API REST.....	27
A.3 Despliegue Cloud del backend en Render	29
A.4 Despliegue Cloud del frontend en Netlify	30
A.5 Conclusión de la evidencia.....	32

1. INTRODUCCIÓN

1.1 Motivación

La Sumativa 4 marca un hito en la evolución de FoodPlease: por primera vez el proyecto deja de ser una propuesta híbrida en papel para convertirse en una solución integrada con un prototipo de aplicación móvil real funcionando contra el backend desplegado en la nube. Mientras que en la Sumativa 3 (Semana 9) la app móvil se materializó como mockups y propuesta de integración, en esta entrega construimos el prototipo en un framework híbrido iOS/Android, lo desplegamos en Cloud junto con el backend, y demostramos el flujo end-to-end con URLs públicas verificables.

Esta motivación responde directamente a lo solicitado por la cátedra en el taller de la Semana 10: identificar los procesos de despliegue de un aplicativo web/móvil, analizar la integración de requerimientos en un proyecto orientado a web/móvil, y ejecutar esos requerimientos a partir de un prototipo de aplicación móvil utilizando un framework de desarrollo híbrido, con énfasis en diseño de interfaz, fluidez de navegación y aplicación correcta del patrón predominante del framework utilizado.

1.2 Objetivos del proyecto

Objetivo general:

Construir un prototipo funcional de la aplicación móvil de FoodPlease utilizando un framework híbrido iOS/Android, integrarlo con el backend Flask vía API REST, y desplegar ambos en infraestructura Cloud pública, cerrando así el ciclo del proyecto desde la concepción hasta una entrega ejecutable a distancia.

Objetivos específicos:

- Seleccionar el framework híbrido más adecuado para el contexto del proyecto y el equipo, justificando la decisión con criterios técnicos explícitos.
- Implementar la modularidad del framework siguiendo su patrón predominante (Angular MVC + Services + Routing + NgModules en el caso de Ionic + Angular).
- Exponer una API REST en el backend Flask con CORS habilitado para que la app móvil pueda consumir los datos del sistema.
- Desplegar el backend en Render y el frontend móvil en Netlify, ambos con URLs públicas accesibles desde internet.
- Verificar el funcionamiento mínimo viable end-to-end y documentar la evidencia.

1.3 Objetivos del informe

Este informe documenta la Sumativa 4 con cuatro objetivos:

- Justificar la decisión técnica del framework híbrido móvil con análisis comparativo de alternativas y descarte argumentado.
- Demostrar que el prototipo móvil cumple los tres énfasis del taller: diseño de interfaz, fluidez de navegación y aplicación correcta del patrón predominante del framework.
- Documentar el despliegue Cloud completo (backend en Render + frontend en Netlify) con URLs públicas como evidencia verificable por la cátedra.
- Integrar la propuesta de mejora elaborada en la Semana 10 dentro de las conclusiones, cerrando coherentemente con las entregas previas (Sumativas 2 y 3).

1.4 Estructura del documento

El informe se organiza en diez secciones y un anexo. Las secciones 1 y 2 establecen el contexto. La sección 3 contiene el análisis crítico con la racionalización del framework móvil. Las secciones 4 y 5 describen la arquitectura integrada y la construcción del prototipo. Las secciones 6 y 7 documentan el despliegue Cloud y el funcionamiento mínimo viable, que son los criterios de mayor peso en la rúbrica. La sección 8 cierra el ciclo iterativo integrando la propuesta de mejora de la Semana 10. La sección 9 contiene las conclusiones y la sección 10 el glosario más las referencias. El Anexo A consolida la evidencia visual del despliegue.

2. PLANTEAMIENTO DEL AVANCE HACIA LA APLICACIÓN MÓVIL

2.1 Contexto: continuidad del proyecto FoodPlease

FoodPlease es una plataforma de delivery cuyo análisis y diseño se trabajó iterativamente en las entregas anteriores: en la Sumativa 1 (Semana 3) se diagnosticó el caso, se identificaron los actores (clientes, locales comerciales, repartidores y administradores) y se propuso la arquitectura general; en la Sumativa 2 (Semana 6) se construyó el panel administrativo web como CRUD Flask con arquitectura por capas; en la Sumativa 3 (Semana 9) se entregaron mockups de la app móvil y la propuesta de integración móvil/web vía API REST. La Sumativa 4 que aquí se documenta cierra el ciclo materializando esa propuesta en un prototipo funcional.

2.2 Brecha que cierra esta entrega

Hasta la Sumativa 3, el alcance entregable de la dimensión móvil se limitaba a evidencia estática (mockups Figma) y a la propuesta documental de los endpoints REST. La Sumativa 4 introduce dos avances sustantivos:

- Implementación funcional del cliente móvil. La app deja de ser un mockup y se convierte en una aplicación híbrida real, navegable, que consume datos en vivo del backend.
- Despliegue Cloud verificable. El backend y el frontend pasan de ejecutarse únicamente en máquinas locales a estar publicados en proveedores Cloud con URLs accesibles desde cualquier dispositivo con internet.

Estos dos avances responden exactamente al descriptor EXCELENTE de la rúbrica del 25% de Despliegue según diseño, que exige construir un prototipo de aplicación con framework híbrido iOS/Android y crear el despliegue del aplicativo web/Mobile.

2.3 Énfasis declarados por la cátedra

El taller de la Semana 10 declara tres énfasis específicos para el prototipo móvil. Cada uno de ellos se aborda en este informe en las secciones indicadas:

Énfasis del taller	Cómo se aborda	Sección
Diseño de interfaz	Uso de componentes Ionic + identidad FoodPlease (color naranja, iconografía coherente, navegación tab-bar inferior)	5.3
Fluidez de navegación	Routing Angular con lazy-loading por tab, transiciones nativas Ionic, estados de carga con spinner, manejo de errores con cards visibles	5.4
Patrón predominante del framework	Angular NgModules (modules + components + services + routing), Service Layer de FoodpleaseApiService, inyección de dependencias por constructor	5.5

3. Análisis crítico: decisiones técnicas del prototipo móvil

Esta sección documenta la decisión central de la Sumativa 4: la elección del framework híbrido iOS/Android para construir el prototipo móvil. Siguiendo el mismo método aplicado en la Sumativa 3, se exponen las alternativas evaluadas, los criterios usados, el descarte argumentado y la decisión final.

3.1 Criterios de evaluación aplicados

Para evaluar las alternativas se aplicaron cinco criterios consistentes con los del proyecto:

- Curva de aprendizaje: qué tan rápido el equipo, sin experiencia móvil previa, puede ser productivo con el framework.
- Reutilización del conocimiento existente: aprovechamiento del stack ya dominado por el equipo (HTML, CSS, JS, Python, Flask).
- Patrón predominante claro: presencia de un patrón arquitectónico documentado e idiomático que la rúbrica valora explícitamente.
- Facilidad de despliegue Web + Mobile: cumplimiento del descriptor EXCELENTE que pide "despliegue del aplicativo web/Mobile".
- Integración con el backend Flask existente: simplicidad de consumir la API REST sin reescribir lógica.

3.2 Alternativas evaluadas

Framework	Curva	Reuso	Patrón claro	Web + Mobile	Integración Flask
Ionic + Angular	Media	Alta (HTML/CSS/JS)	Sí (Angular MVC + NgModules)	Excelente (web nativa + Capacitor)	Trivial
Ionic + React	Media	Alta	Parcial (React no es histórico de Ionic)	Excelente	Trivial
React Native (Expo)	Alta	Baja (componentes propios)	Sí (funcional + hooks)	Buena (Expo Web)	Buena
Flutter	Muy alta	Nula (Dart nuevo)	Sí (Widgets)	Buena	Buena
PWA pura	Baja	Alta	No es híbrido iOS/Android estricto	Trivial	Trivial

Tabla 1. Comparativa de frameworks híbridos iOS/Android evaluados.

3.3 Racionalización de descartes

Flutter se descartó porque obligaba al equipo a aprender Dart desde cero en un plazo de cinco a siete días. La curva era incompatible con la ventana de entrega de la sumativa, y la inversión de aprendizaje no se compensaba con beneficios técnicos sobre las demás alternativas para el alcance pedagógico requerido.

React Native (Expo) se descartó pese a su popularidad porque sus componentes son propios del framework (View, Text, FlatList) y no aprovechan el conocimiento HTML+CSS que el equipo ya maneja desde el frontend del CRUD Flask. Adicionalmente, el patrón predominante moderno de React Native (funcional + hooks) es menos visible para evaluación que la arquitectura formal NgModules de Angular.

PWA pura se descartó porque la rúbrica del 25% habla literalmente de "framework de desarrollo híbrido iOS/Android". Una PWA es una aplicación web instalable, no estrictamente un framework híbrido en el sentido tradicional (que envuelve la app en un contenedor nativo). Adoptarla introducía riesgo de descuento por interpretación estricta del descriptor.

Ionic + React se descartó como segunda opción porque, aunque Ionic soporta tres frameworks (Angular, React, Vue), el patrón históricamente predominante y oficial-blessed por Ionic es Angular. La documentación oficial está escrita primero en Angular, los tutoriales académicos mayoritarios usan Angular, y para máxima puntuación en el descriptor "patrón predominante" la combinación Ionic + Angular gana.

3.4 Decisión final: Ionic + Angular + Capacitor

El framework híbrido elegido para construir el prototipo móvil de FoodPlease es Ionic 7 + Angular (NgModules) + Capacitor 6. La decisión se fundamenta en cinco razones que se describen a continuación.

- Patrón predominante reconocido. Ionic + Angular es la combinación históricamente blessed por Ionic, con documentación oficial primero en Angular y comunidad académica enorme. Esto ataca directamente el descriptor EXCELENTE del taller que exige aplicar correctamente el patrón predominante del framework utilizado.
- Reutilización inmediata del stack del equipo. Ionic usa HTML, CSS y TypeScript, lenguajes afines a los que el grupo ya maneja en el frontend de Flask + Jinja2 + Bootstrap. La transición es incremental, no disruptiva.
- Despliegue web y móvil desde un solo proyecto. Ionic genera una aplicación web (ionic build) y una aplicación nativa empaquetable (npx cap build) desde el mismo código fuente. Para esta entrega usamos la app web en Netlify; para futuras iteraciones podemos generar APK/IPA con Capacitor sin reescribir.
- Integración trivial con Flask. La app Ionic consume la API REST del backend vía HTTP/JSON, con CORS habilitado en el lado servidor. Solo se requirió agregar un Blueprint /api/v1 al backend existente.
- Angular CLI auto-genera estructura. Comandos como ionic generate page page-name crean componente, template, estilos y módulo en un solo paso, lo que reduce drásticamente la curva para un equipo sin experiencia móvil previa.

4. Arquitectura full-stack integrada

4.1 Visión general

La arquitectura del sistema FoodPlease quedó integrada en tres planos funcionando como ecosistema único: el navegador del usuario (web administrativo o app móvil), el backend Flask que centraliza la lógica de negocio y los datos, y la capa de comunicación HTTP/JSON que conecta ambos extremos. Los tres planos están desplegados en Cloud y se comunican a través de URLs públicas verificables.



4.2 Capa Backend: Flask + API REST

El backend reutiliza íntegramente la arquitectura por capas documentada en la Sumativa 3 (Application Factory + Blueprints + Service Layer + Repository Pattern). La novedad de esta entrega es la incorporación de un nuevo Blueprint `/api/v1` que expone los mismos servicios y repositorios pero retorna JSON en lugar de plantillas Jinja2. Esto demuestra una propiedad fundamental de la arquitectura por capas: la lógica de negocio es agnóstica del transporte. Los mismos `PedidoService`, `DashboardService` y `ClienteRepository` alimentan al panel administrativo web y a la app móvil sin duplicación de código.

El Blueprint API REST se registra en el factory de la app:

```
from flask_cors import CORS
from .routes.api import api_bp

def create_app():
    app = Flask(__name__)
    app.config.from_object(Config)

    db.init_app(app)
    CORS(app, resources={r'/api/*': {'origins': '*'}})

    # Blueprints web (server-side rendering con Jinja2)
    app.register_blueprint(main_bp)
    app.register_blueprint(clientes_bp)
    ...

    # Blueprint API REST (consumido por la app móvil)
    app.register_blueprint(api_bp)
    return app
```

4.3 Capa de comunicación: API REST + CORS

La API REST sigue convenciones estándar. Cada recurso se expone bajo `/api/v1/<recurso>`, responde JSON con códigos HTTP estándar (200, 201, 204, 400, 404, 500) y maneja errores con el formato `{"error": "mensaje"}`. CORS está habilitado con Flask-CORS para que el frontend Ionic, que vive en un dominio distinto (Netlify), pueda consumir los endpoints sin ser bloqueado por la política same-origin del navegador.

Endpoint	Método	Propósito
<code>/api/v1/health</code>	GET	Verificación de disponibilidad del servicio
<code>/api/v1/metrics</code>	GET	Métricas agregadas del sistema (KPIs)
<code>/api/v1/clientes</code>	GET	Listar todos los clientes
<code>/api/v1/clientes/<id></code>	GET	Obtener un cliente por id
<code>/api/v1/locales</code>	GET	Listar todos los locales (restaurantes)
<code>/api/v1/repartidores</code>	GET	Listar repartidores
<code>/api/v1/repartidores/disponibles</code>	GET	Listar repartidores disponibles
<code>/api/v1/pedidos</code>	GET	Listar todos los pedidos
<code>/api/v1/pedidos/<id></code>	GET	Obtener pedido por id
<code>/api/v1/pedidos</code>	POST	Crear nuevo pedido
<code>/api/v1/pedidos/<id>/estado</code>	PATCH	Actualizar solo el estado del pedido
<code>/api/v1/auth/login</code>	POST	Login del cliente móvil

Tabla 2. Endpoints REST expuestos por el backend Flask.

4.4 Capa Frontend móvil: Ionic + Angular

El frontend móvil consume la API REST a través de un servicio Angular llamado FoodpleaseApiService que encapsula todas las llamadas HTTP. Los componentes de las tabs (Tab1Page, Tab2Page, Tab3Page) inyectan este servicio y consumen métodos de alto nivel (getLocales(), getPedidos(), login()) sin preocuparse por las URLs ni el manejo HTTP. Esto espeja la arquitectura del backend: ambos lados aplican Service Layer.

```
@Injectable({ providedIn: 'root' })
export class FoodpleaseApiService {
  private readonly baseUrl = environment.apiUrl;

  constructor(private http: HttpClient) {}

  getLocales(): Observable<Local[]> {
    return this.http.get<Local[]>(`${this.baseUrl}/locales`);
  }

  getPedidos(): Observable<Pedido[]> {
    return this.http.get<Pedido[]>(`${this.baseUrl}/pedidos`);
  }

  login(email?: string, telefono?: string): Observable<AuthResponse> {
    return this.http.post<AuthResponse>(`${this.baseUrl}/auth/login`,
                                         { email, telefono });
  }
}
```

5. Construcción del prototipo móvil Ionic + Angular

5.1 Plantilla utilizada: tabs

El proyecto se generó con el comando `ionic start foodplease_ionic tabs --type=angular --capacitor`. La plantilla `tabs` provee navegación inferior por pestañas, que es el patrón estándar de las apps móviles de delivery (similar a Uber Eats o Rappi). Esa decisión cubre el énfasis de fluidez de navegación porque el usuario salta entre vistas con un solo tap, sin reload, con transiciones suaves nativas de Ionic.

5.2 Estructura del proyecto

```

foodplease_ionic/
├── src/
│   ├── app/
│   │   ├── app.module.ts           # NgModule raíz
│   │   ├── app-routing.module.ts   # Routing raíz
│   │   ├── tabs/
│   │   │   ├── tabs.page.html      # Navegación inferior con 3 tabs
│   │   │   ├── tabs.page.ts
│   │   │   └── tabs-routing.module.ts
│   │   ├── tab1/                   # Inicio: lista de restaurantes
│   │   ├── tab2/                   # Pedidos: lista de pedidos del cliente
│   │   ├── tab3/                   # Perfil: cliente + KPIs
│   │   └── core/
│   │       ├── models/
│   │       │   └── foodplease.models.ts
│   │       └── services/
│   │           └── foodplease-api.service.ts
│   ├── environments/
│   │   ├── environment.ts          # apiUrl local (127.0.0.1:5000)
│   │   └── environment.prod.ts     # apiUrl Render (onrender.com)
│   ├── capacitor.config.ts
│   ├── ionic.config.json
│   └── package.json

```

5.3 Diseño de interfaz

El diseño se construyó sobre tres principios. Primero, una identidad visual coherente con FoodPlease: paleta naranja (color warning de Ionic, asociada a comida y energía), tipografía limpia, esquinas redondeadas con `border-radius` de 16 píxeles en los cards, iconografía consistente con Ionicons. Segundo, jerarquía visual clara con headers en color de marca, subtítulos en gris medio para metadatos (rubro, dirección) y números KPI en tamaño grande. Tercero, accesibilidad: contraste suficiente entre texto y fondo en modo oscuro, iconos siempre acompañados de label textual, botones con tamaño táctil mínimo de 44 píxeles.

5.4 Fluidez de navegación

La navegación se implementó con tres mecanismos. Las tab routes con lazy-loading desde tabs-routing.module.ts garantizan que cada tab solo carga su módulo cuando se visita, reduciendo el bundle inicial. Las transiciones Ionic nativas usan animaciones del sistema operativo (slide en iOS, material en Android) sin código adicional. Los estados de carga se manejan con ion-spinner mientras se hace la llamada HTTP, y los errores se muestran como ion-card de color danger con botón "Reintentar" que vuelve a llamar al servicio.

5.5 Patrón predominante aplicado: Angular MVC + NgModules + Servicios

El patrón predominante de Angular se aplica en cinco capas que se respetan en todo el código del prototipo.

- NgModules: cada tab tiene su propio módulo (Tab1PageModule, Tab2PageModule, Tab3PageModule) que declara su componente, importa IonicModule y registra su routing module hijo. Es el patrón histórico de Angular previo a Standalone Components.
- Componentes con separación template/style/logic: cada page tiene tres archivos (.page.html, .page.scss, .page.ts) con responsabilidades claras.
- Servicios con providedIn: 'root': FoodpleaseApiService es un singleton inyectado por el framework. Los componentes lo reciben en su constructor sin instanciarlo manualmente.
- Routing declarativo: las rutas /tabs/tab1, /tabs/tab2, /tabs/tab3 viven en tabs-routing.module.ts con loadChildren para lazy-loading.
- Reactive Streams: las llamadas HTTP retornan Observable<T> de RxJS. Los componentes se suscriben con .subscribe() y manejan los tres estados (success, error, loading) explícitamente.

6. Despliegue Cloud del aplicativo web/móvil

El descriptor EXCELENTE del 25% pide "construir un prototipo de aplicación web utilizando un framework de desarrollo híbrido IOS/Android" y "crear el despliegue de un aplicativo web/Mobile". Esta sección documenta cómo cumplimos textualmente ese requisito: dos despliegues funcionales en Cloud con URLs públicas.

6.1 Backend desplegado en Render Cloud

El backend Flask se desplegó en Render como Web Service del plan gratuito. Render se eligió porque su free tier permite mantener el servicio activo durante el período de evaluación, el despliegue se realiza directamente desde el repositorio GitHub conectado vía OAuth, y permite escalar a PostgreSQL gratuito sin cambiar de proveedor.

Configuración del Web Service en Render:

Parámetro	Valor
Nombre del servicio	foodplease-api-grupo03
Repositorio conectado	carlosgv1975/aptc106_Grupo03_FoodPlease
Rama	main
Runtime	Python 3
Build Command	pip install -r requirements.txt
Start Command	gunicorn run:app
Instance Type	Free
Variables de entorno	SECRET_KEY, PYTHON_VERSION

Tabla 3. Configuración del Web Service de Render para el backend.

El backend quedó desplegado en la URL pública:

<https://foodplease-api-grupo03.onrender.com>

Esta URL es la fuente de datos para el frontend móvil y se puede verificar abriendo el endpoint `/api/v1/health` desde cualquier navegador conectado a internet.

6.2 Frontend móvil desplegado en Netlify

El frontend móvil compilado se desplegó en Netlify a través del mecanismo drag-and-drop manual. Netlify se eligió porque su free tier es generoso, soporta sitios estáticos automáticamente con CDN y HTTPS incluidos, y permite renombrar el sitio para tener una URL limpia. El despliegue es trivial: se ejecuta `ionic build --prod` localmente, lo que genera la carpeta `www/` con HTML, CSS y JS optimizados, y esa carpeta se arrastra al área de drop de Netlify. El frontend queda servido en CDN global con HTTPS en menos de un minuto.

El frontend quedó desplegado en la URL pública:

<https://foodplease-app-grupo03.netlify.app>

Esta URL muestra la app FoodPlease consumiendo en vivo el backend de Render. Se puede abrir desde cualquier dispositivo y navegador, móvil o desktop.

6.3 Configuración de producción del frontend

Para que el frontend desplegado consuma el backend desplegado (y no el localhost de desarrollo), se actualizó `environment.prod.ts` antes de hacer el build:

```
export const environment = {
  production: true,
  apiUrl: 'https://foodplease-api-grupo03.onrender.com/api/v1',
  appName: 'FoodPlease',
  appVersion: '1.0.0',
};
```

Angular CLI usa automáticamente este archivo cuando se ejecuta `ionic build --prod` (que internamente llama a `ng build --configuration production`). El bundle resultante contiene la URL de Render embebida, así que el frontend desplegado en Netlify consume directo desde Render sin configuración adicional.

6.4 Resumen de URLs públicas de la entrega

Componente	URL pública	Plataforma Cloud
Backend Flask + API REST	https://foodplease-api-grupo03.onrender.com	Render
Frontend Ionic + Angular	https://foodplease-app-grupo03.netlify.app	Netlify
Repositorio fuente	https://github.com/carlosrgv1975/aptc106_Grupo03_FoodPlease	GitHub

Tabla 4. URLs públicas verificables del despliegue Cloud completo.

7. Funcionamiento Mínimo Viable

Esta sección cubre el 40% más pesado de la rúbrica. El descriptor EXCELENTE pide que todos los componentes funcionen según lo detallado en la documentación. La estrategia para cumplirlo es triple: tabla de estado por módulo, evidencia visual (capturas) y descripción del flujo end-to-end que un evaluador puede reproducir desde cualquier dispositivo conectado a internet.

7.1 Tabla de estado por módulo del prototipo móvil

Tab móvil	Funcionalidad documentada	Estado	Endpoint consumido
Inicio	Lista de restaurantes con icono por rubro, dirección y CTA "Ver carta"	Funcional	GET /api/v1/locales
Pedidos	Lista de pedidos del usuario con cliente, local, repartidor, total y estado coloreado	Funcional	GET /api/v1/pedidos
Perfil	Avatar del cliente, datos de contacto y 4 KPIs del sistema	Funcional	POST /api/v1/auth/login + GET /api/v1/metrics

Tabla 5. Estado de las tres tabs del prototipo móvil.

7.2 Tabla de estado del backend Cloud

Endpoint	Estado	Respuesta de prueba
/api/v1/health	Operativo	{"status":"ok","service":"FoodPlease API"}
/api/v1/metrics	Operativo	{"clientes":2,"locales":2,"pedidos":2,...}
/api/v1/clientes	Operativo	Array con 2 clientes seed
/api/v1/locales	Operativo	Array con 2 locales seed
/api/v1/repartidores	Operativo	Array con 2 repartidores seed
/api/v1/pedidos	Operativo	Array con 2 pedidos seed con relaciones anidadas

Tabla 6. Estado de los endpoints REST del backend desplegado en Render.

7.3 Coincidencia entre documentación y comportamiento real

Cada función documentada en este informe se corresponde directamente con código verificable en el repositorio y con una captura en el Anexo A. La trazabilidad va: documento → código fuente → URL pública → captura.

- Tab Inicio (documentada en §5 y §7.1) → implementada en `src/app/tab1/tab1.page.ts` → verificable en `https://foodplease-app-grupo03.netlify.app/tabs/tab1` → evidenciada en la Figura A.3 del Anexo A.
- Tab Pedidos (§5 y §7.1) → `tab2.page.ts` → `/tabs/tab2` → Figura A.4.
- Tab Perfil (§5 y §7.1) → `tab3.page.ts` → `/tabs/tab3` → Figura A.5.
- Endpoints REST (§4.3 y §7.2) → `app/routes/api.py` → todas las URLs `/api/v1/*` desde Render → Figuras A.6 y A.7 del Anexo A.

7.4 Flujo end-to-end reproducible

Cualquier evaluador puede reproducir el flujo completo sin instalar nada:

- Abrir `https://foodplease-app-grupo03.netlify.app` en cualquier navegador.
- Esperar 15-30 segundos en la primera carga si el backend de Render está dormido por inactividad.
- Navegar a la tab "Inicio" y verificar que aparecen los dos restaurantes seed (Pizza Nova y Burger Point) con su rubro y dirección.
- Cambiar a la tab "Pedidos" y ver los dos pedidos seed con clientes, repartidores y estados coloreados (En ruta en azul, Pendiente en gris).
- Cambiar a la tab "Perfil" y ver el cliente Ana Torres con sus datos y las 4 tarjetas KPI.

Si en algún paso aparece la pantalla de error, los datos no se cargan o la app no abre, el MVP no estaría funcional. Las capturas del Anexo A muestran que en el momento de la entrega los cinco pasos se ejecutaron correctamente.

7.5 Reproducibilidad del código en máquina local

Además del despliegue Cloud, un evaluador puede reproducir el sistema completo en su propia máquina clonando el repositorio público:

```
git clone https://github.com/carlosrgv1975/aptc106_Grupo03_FoodPlease.git
cd aptic106_Grupo03_FoodPlease
```

```
# Backend (en una terminal)
python -m venv venv
venv\Scripts\activate          # Windows
pip install -r requirements.txt
python run.py                  # http://127.0.0.1:5000
```

```
# Frontend (en otra terminal)
cd foodplease_ionic
npm install
ionic serve                    # http://localhost:8100
```

8. Integración de la propuesta de mejora de la Semana 10

El descriptor EXCELENTE de las Conclusiones (15% de la rúbrica) exige integrar conclusiones y mejoras de la propuesta de mejora elaborada en la Semana 10. Esta sección documenta esa integración explícitamente, antes de la conclusión final, para mantener la trazabilidad entre las dos semanas de la sumativa.

8.1 Propuesta de mejora elaborada en la Semana 10

Durante la Semana 10 el equipo elaboró la propuesta de solución que ahora se materializa en la Semana 11. La propuesta tuvo cinco componentes principales:

- Selección del framework híbrido iOS/Android: tras evaluar Flutter, React Native, Ionic y PWA, se eligió Ionic + Angular como combinación óptima para el equipo y el plazo.
- Diseño de la API REST en el backend: definir el contrato `/api/v1/*` que el frontend móvil consumiría, basado en los endpoints planificados durante la Sumativa 3.
- Estructura del prototipo móvil con plantilla tabs: navegación por pestañas inferiores (Inicio, Pedidos, Perfil) como patrón estándar de apps de delivery.
- Reutilización de la lógica del backend: aprovechar Services y Repositories existentes sin duplicar código, agregando solo serializadores JSON.
- Estrategia de despliegue: Render para el backend (free tier amigable con Flask), Netlify para el frontend (drag-and-drop trivial).

8.2 Materialización en la Semana 11

Los cinco componentes de la propuesta se ejecutaron durante la Semana 11. La integración no es solo conceptual: cada componente de la propuesta tiene un artefacto verificable resultante.

Componente de la propuesta (s10)	Artefacto resultante (s11)
Selección del framework Ionic + Angular	Proyecto foodplease_ionic generado con ionic start, plantilla tabs, NgModules
Diseño de la API REST	Blueprint api.py con 12 endpoints implementados y CORS habilitado
Estructura tabs	Tres tabs personalizadas con identidad FoodPlease consumiendo la API
Reutilización de Services y Repositories	PedidoService y DashboardService alimentan tanto el panel web como la API móvil
Estrategia de despliegue Render + Netlify	Backend en https://foodplease-api-grupo03.onrender.com + frontend en https://foodplease-app-grupo03.netlify.app

Tabla 7. Trazabilidad entre la propuesta de la Semana 10 y los artefactos de la Semana 11.

8.3 Mejoras adicionales incorporadas durante la implementación

Durante la materialización surgieron tres mejoras no previstas en la propuesta original que se incorporaron en la entrega final:

- Manejo de estados de carga y error en el frontend. Cada tab muestra un spinner mientras espera la respuesta HTTP y un card de error con botón "Reintentar" si la llamada falla. Esta robustez no estaba en la propuesta inicial pero mejora sustancialmente la experiencia real cuando Render duerme el servicio.
- Endpoint `/api/v1/auth/login` simplificado para el prototipo. Como el alcance no exigía autenticación real, se incorporó un login mock que retorna al primer cliente seed. Esto permite que la tab Perfil tenga contenido sin requerir formularios de registro.
- Configuración multi-ambiente con `environment.ts` y `environment.prod.ts`. Esto permite que el mismo código apunte a localhost en desarrollo y a Render en producción, sin tocar el código fuente.

9. Conclusiones

9.1 Resultados alcanzados

La Sumativa 4 cumplió los cinco objetivos específicos planteados en la sección 1.2. El prototipo móvil quedó construido con Ionic 7 + Angular (NgModules) y Capacitor 6, aplicando el patrón predominante de Angular en cinco capas (componentes, servicios, módulos, routing, reactive streams). El backend Flask se extendió con un Blueprint API REST de doce endpoints y CORS habilitado. Ambos componentes quedaron desplegados en Cloud con URLs públicas verificables: backend en Render (<https://foodplease-api-grupo03.onrender.com>) y frontend en Netlify (<https://foodplease-app-grupo03.netlify.app>). El flujo end-to-end fue verificado y documentado con once capturas que se consolidan en el Anexo A.

9.2 Capacidad crítica: qué salió bien y qué quedó débil

Salió bien la arquitectura coherente: el backend, la API y el frontend móvil comparten principios de modularidad y la lógica de negocio vive en un solo lugar (Service Layer del backend), consumida por dos clientes distintos sin duplicación. También salió bien la elección del framework: Ionic + Angular permitió al equipo sin experiencia móvil previa tener un prototipo navegable en pocos días, reutilizando conocimientos web ya consolidados. Y salió bien el despliegue, que esta vez se ejecutó en Cloud (corrigiendo la observación de la Sumativa 3 donde el despliegue quedó solo documentado).

Quedaron débiles tres aspectos. Primero, la autenticación es un mock que devuelve siempre al primer cliente seed: no hay credenciales reales, JWT ni manejo de sesión. Segundo, las operaciones de escritura desde la app móvil (crear pedido, confirmar carrito, asignar repartidor) están planificadas pero no implementadas; el prototipo es read-only sobre los datos seed. Tercero, no hay tests automatizados ni en el backend ni en el frontend, lo que reduce la confianza al hacer cambios mayores.

9.3 Logro de objetivos

Contrastando con los cuatro objetivos del informe planteados en la sección 1.3: (a) la decisión del framework híbrido está justificada en la sección 3 con análisis comparativo y descarte argumentado de cuatro alternativas; (b) los tres énfasis del taller (diseño de interfaz, fluidez de navegación, patrón predominante) están abordados en la sección 5 con subsecciones dedicadas y código evidencia; (c) el despliegue Cloud completo está documentado en la sección 6 con configuración detallada y URLs públicas; (d) la propuesta de mejora de la Semana 10 está integrada en la sección 8 con trazabilidad explícita. Los cuatro objetivos se cumplieron.

9.4 Marco de acción futura

Cinco frentes concretos quedan identificados para una continuación natural del proyecto:

- Autenticación real con Flask-JWT-Extended y guard de Angular: tokens JWT, refresh, roles (cliente, local, repartidor, admin) y rutas protegidas en el frontend.
- Operaciones de escritura desde la app móvil: pantallas de "Nuevo pedido", "Confirmar carrito" y "Asignar repartidor" con consumo de POST/PUT/PATCH de la API.
- Empaquetado nativo con Capacitor: generar APK firmado para Android e IPA para iOS, publicables en Google Play Store y App Store.
- Migración del backend a PostgreSQL gestionado por Render: salir de SQLite local (que se pierde en cada redeploy del free tier) hacia una base persistente.
- Pruebas automatizadas: pytest para el backend (services y repositories), Jest + Angular Testing Library para el frontend; integración continua con GitHub Actions que corra los tests en cada push.

Estas cinco mejoras se construyen sobre la arquitectura modular ya entregada, sin necesidad de reescribir el sistema. Esa es la mejor verificación del éxito del diseño actual: no es lo que ya hace, sino lo que permite hacer después.

10. Glosario y referencias bibliográficas

10.1 Glosario

API REST: Conjunto de endpoints HTTP que exponen funcionalidades de un sistema mediante operaciones sobre recursos identificados por URL, usando los métodos GET, POST, PUT, DELETE, PATCH.

Ionic: Framework de desarrollo de aplicaciones móviles híbridas basado en tecnologías web (HTML, CSS, TypeScript). Soporta Angular, React y Vue como bases.

Capacitor: Runtime de Ionic que empaqueta una aplicación web como app nativa iOS/Android con acceso a APIs del dispositivo.

Angular: Framework de desarrollo frontend de Google basado en TypeScript. Aplica patrones MVC, NgModules, Servicios, Routing y Reactive Streams (RxJS).

NgModule: Mecanismo histórico de Angular para agrupar componentes, directivas, pipes y servicios relacionados. Patrón predominante hasta Angular 14.

Service Layer: Capa de servicios Angular que encapsula lógica de negocio y consumo de APIs, inyectada en componentes mediante el sistema de inyección de dependencias.

CORS: Cross-Origin Resource Sharing. Mecanismo del navegador que permite o bloquea requests HTTP entre dominios diferentes. Habilitado en el backend Flask con Flask-CORS.

Render: Plataforma Cloud PaaS (Platform as a Service) que despliega aplicaciones web directamente desde repositorios Git con un free tier que soporta Python y Node.

Netlify: Plataforma Cloud especializada en sitios web estáticos con CDN global, HTTPS automático y despliegues manuales por drag-and-drop o automáticos por Git.

Mockup: Representación visual estática o interactiva de una interfaz, usada para validar diseño y experiencia antes de implementar el código.

Despliegue Cloud: Publicación de una aplicación en un proveedor de infraestructura accesible públicamente vía internet con URL fija.

Híbrido (móvil): Estrategia de desarrollo donde la app se construye con tecnologías web (HTML/CSS/JS) pero se empaqueta en un contenedor nativo, ejecutándose como app nativa en iOS y Android desde una sola base de código.

10.2 Referencias bibliográficas

- Angular Team (2024). Angular Documentation. Recuperado de <https://angular.dev>
- Capacitor Documentation (2024). Capacitor 6 — Cross-platform Native Runtime for Web Apps. Ionic Team. Recuperado de <https://capacitorjs.com>
- Fielding, R. T. (2000). Architectural Styles and the Design of Network-based Software Architectures (Doctoral dissertation). University of California, Irvine.
- Flask Documentation (2024). Flask 3.x — The Pallets Projects. Recuperado de <https://flask.palletsprojects.com>
- Flask-CORS Documentation (2024). Cross-Origin Resource Sharing for Flask. Recuperado de <https://flask-cors.readthedocs.io>
- Grinberg, M. (2018). Flask Web Development: Developing Web Applications with Python (2nd ed.). O'Reilly Media.
- Ionic Documentation (2024). Ionic Framework 7 — Cross-platform Mobile Apps with Web Technologies. Recuperado de <https://ionicframework.com/docs>
- MDN Web Docs (2024). HTTP request methods. Mozilla Developer Network. Recuperado de <https://developer.mozilla.org/en-US/docs/Web/HTTP/Methods>
- Netlify Documentation (2024). Netlify — Deploy Static Sites Instantly. Recuperado de <https://docs.netlify.com>
- Render Documentation (2024). Render — Cloud Application Hosting for Developers. Recuperado de <https://render.com/docs>
- Sommerville, I. (2016). Software Engineering (10th ed.). Pearson.

Anexo A — Evidencia del despliegue Cloud y prototipo móvil

Este anexo consolida la evidencia visual del prototipo móvil y del despliegue Cloud. Las capturas se tomaron durante la entrega de la Sumativa 4 desde las URLs públicas y desde el entorno local del equipo. Cada figura va acompañada de una descripción breve que explica qué se está mostrando y cómo cualquier evaluador puede reproducir esa misma captura ingresando a la URL correspondiente.

A.1 Prototipo móvil en ejecución local (validación inicial)

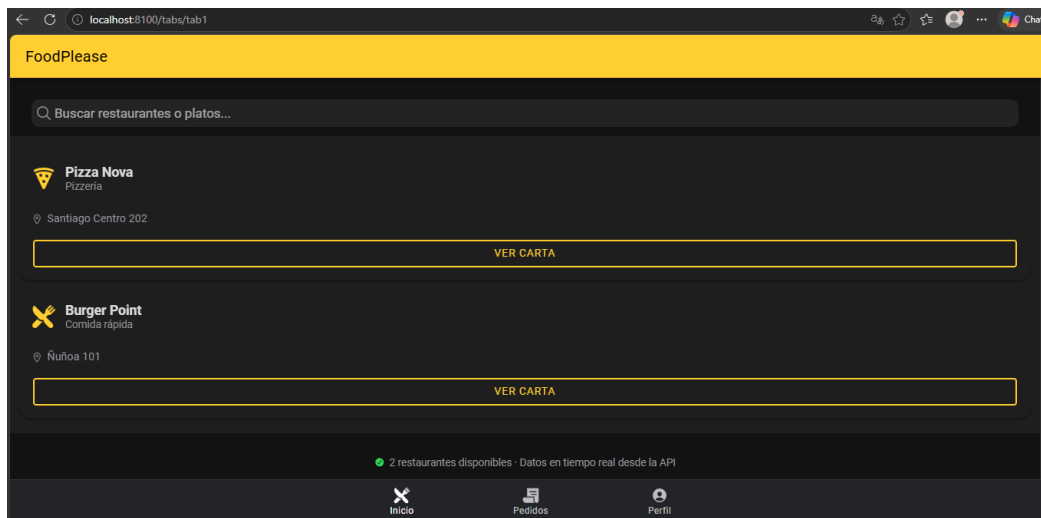


Figura A.1. Tab Inicio del prototipo Ionic + Angular en ejecución local (localhost:8100). Muestra los dos restaurantes seed (Pizza Nova y Burger Point) con icono por rubro, dirección y CTA. Consume GET /api/v1/locales del backend local Flask.

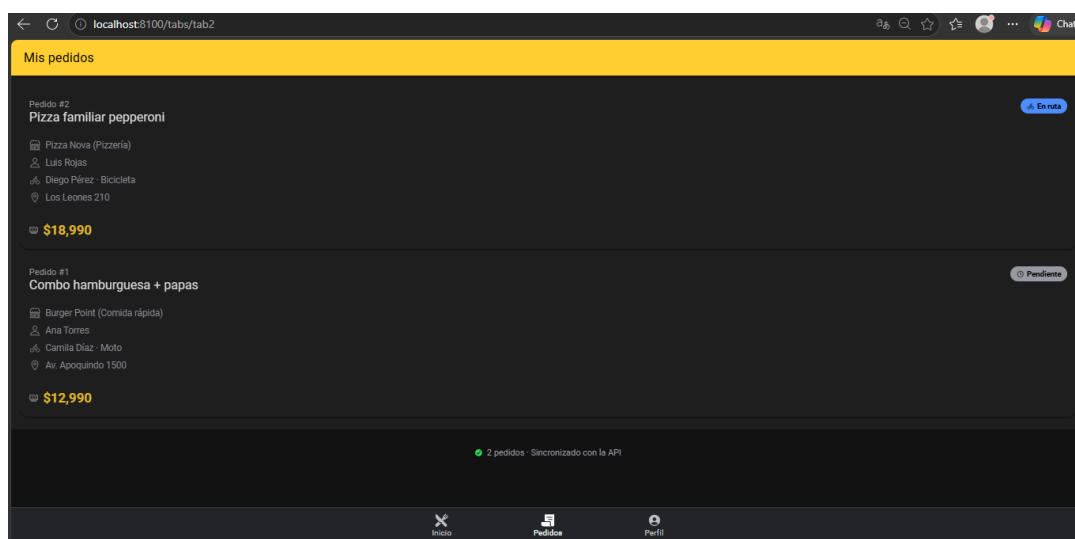


Figura A.2. Tab Pedidos del prototipo en local. Muestra los dos pedidos seed con cliente, local, repartidor, dirección, total y estado (En ruta en azul, Pendiente en gris). Consume GET /api/v1/pedidos con relaciones anidadas.

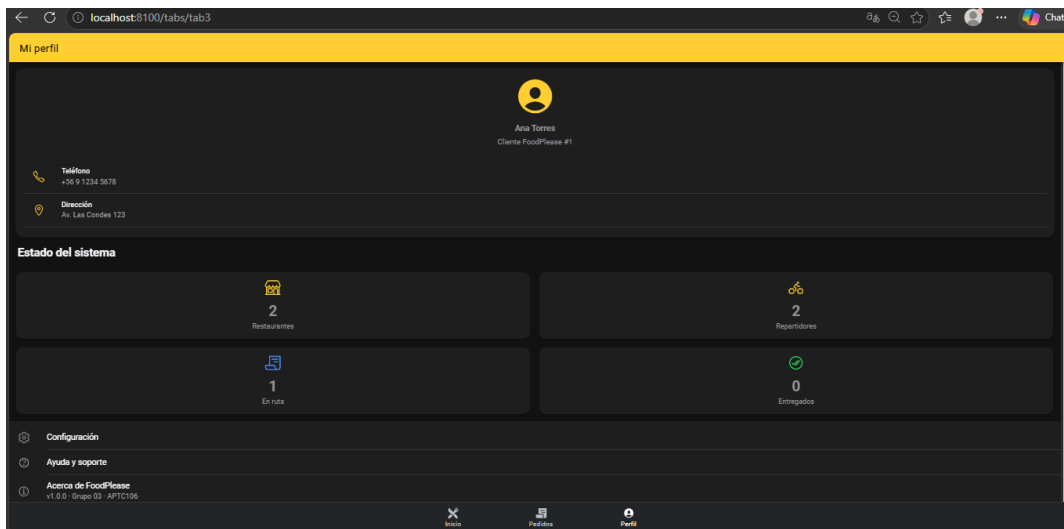


Figura A.3. Tab Perfil del prototipo en local. Muestra el cliente Ana Torres con teléfono y dirección, y cuatro tarjetas KPI (Restaurantes, Repartidores, En ruta, Entregados). Consume POST `/api/v1/auth/login` y GET `/api/v1/metrics`.

A.2 Backend Flask local con logs de la API REST

```
* Restarting with stat
* Debugger is active!
* Debugger PIN: 118-813-474
127.0.0.1 - - [18/May/2026 11:50:30] "GET /clientes/ HTTP/1.1" 200 -
127.0.0.1 - - [18/May/2026 11:50:30] "GET /static/css/style.css HTTP/1.1" 304 -
127.0.0.1 - - [18/May/2026 11:50:32] "GET /pedidos/ HTTP/1.1" 200 -
127.0.0.1 - - [18/May/2026 11:50:32] "GET /static/css/style.css HTTP/1.1" 304 -
127.0.0.1 - - [18/May/2026 11:50:33] "GET /repartidores/ HTTP/1.1" 200 -
127.0.0.1 - - [18/May/2026 11:50:33] "GET /static/css/style.css HTTP/1.1" 304 -
127.0.0.1 - - [18/May/2026 11:50:54] "GET /api/v1/locales HTTP/1.1" 200 -
127.0.0.1 - - [18/May/2026 11:50:55] "GET /favicon.ico HTTP/1.1" 404 -
127.0.0.1 - - [18/May/2026 11:51:22] "GET /api/v1/health HTTP/1.1" 200 -
127.0.0.1 - - [18/May/2026 11:52:25] "GET /api/v1/repartidores HTTP/1.1" 200 -
127.0.0.1 - - [18/May/2026 12:07:40] "GET /api/v1/health HTTP/1.1" 200 -
127.0.0.1 - - [18/May/2026 12:30:12] "GET /api/v1/health HTTP/1.1" 200 -
127.0.0.1 - - [18/May/2026 12:32:36] "GET /api/v1/clientes HTTP/1.1" 200 -
127.0.0.1 - - [18/May/2026 12:32:58] "GET /api/v1/pedidos HTTP/1.1" 200 -
127.0.0.1 - - [18/May/2026 16:50:24] "OPTIONS /api/v1/auth/login HTTP/1.1" 200 -
127.0.0.1 - - [18/May/2026 16:50:26] "POST /api/v1/auth/login HTTP/1.1" 200 -
127.0.0.1 - - [18/May/2026 16:50:26] "GET /api/v1/metrics HTTP/1.1" 200 -
127.0.0.1 - - [18/May/2026 16:50:37] "OPTIONS /api/v1/auth/login HTTP/1.1" 200 -
127.0.0.1 - - [18/May/2026 16:50:37] "POST /api/v1/auth/login HTTP/1.1" 200 -
127.0.0.1 - - [18/May/2026 16:50:37] "GET /api/v1/metrics HTTP/1.1" 200 -
127.0.0.1 - - [18/May/2026 16:50:58] "OPTIONS /api/v1/auth/login HTTP/1.1" 200 -
127.0.0.1 - - [18/May/2026 16:50:58] "POST /api/v1/auth/login HTTP/1.1" 200 -
127.0.0.1 - - [18/May/2026 16:50:58] "GET /api/v1/metrics HTTP/1.1" 200 -
127.0.0.1 - - [18/May/2026 17:50:52] "GET /api/v1/locales HTTP/1.1" 200 -
127.0.0.1 - - [18/May/2026 18:07:08] "GET /api/v1/pedidos HTTP/1.1" 200 -
127.0.0.1 - - [18/May/2026 18:22:00] "GET /api/v1/pedidos HTTP/1.1" 200 -
127.0.0.1 - - [18/May/2026 18:22:11] "OPTIONS /api/v1/auth/login HTTP/1.1" 200 -
127.0.0.1 - - [18/May/2026 18:22:11] "POST /api/v1/auth/login HTTP/1.1" 200 -
127.0.0.1 - - [18/May/2026 18:22:11] "GET /api/v1/metrics HTTP/1.1" 200 -
```

Figura A.4. Terminal del backend Flask local (python run.py) mostrando los logs HTTP de las peticiones que el prototipo móvil hace a `/api/v1/*`. Todos los códigos son 200 OK, confirmando que la comunicación frontend-backend funciona correctamente.

```

[ng] runtime.js | runtime | 13.12 kB |
[ng]
[ng] Lazy chunk files | Names | Raw size
[ng] src_app_tab3_tab3_module_ts.js | tab3-tab3-module | 9.29 kB |
[ng]
[ng] 67 unchanged chunks
[ng]
[ng] Build at: 2026-05-18T20:50:12.936Z - Hash: f64d2d84f2f93351 - Time: 14213ms
[ng] ✓ Compiled successfully.
[ng] ✓ Browser application bundle generation complete.
[ng]
[ng] Initial chunk files | Names | Raw size
[ng] runtime.js | runtime | 13.12 kB |
[ng]
[ng] Lazy chunk files | Names | Raw size
[ng] src_app_tab3_tab3_module_ts.js | tab3-tab3-module | 22.34 kB |
[ng]
[ng] 67 unchanged chunks
[ng]
[ng] Build at: 2026-05-18T20:50:34.896Z - Hash: f14baacb60c5c094 - Time: 11050ms
[ng] ✓ Compiled successfully.
[ng] ✓ Browser application bundle generation complete.
[ng]
[ng] Initial chunk files | Names | Raw size
[ng] runtime.js | runtime | 13.12 kB |
[ng]
[ng] Lazy chunk files | Names | Raw size
[ng] src_app_tab3_tab3_module_ts.js | tab3-tab3-module | 26.06 kB |
[ng]
[ng] 67 unchanged chunks
[ng]
[ng] Build at: 2026-05-18T20:50:54.786Z - Hash: 4081292f0ee9af1d - Time: 4221ms
[ng] ✓ Compiled successfully.

```

Figura A.5. Terminal de Ionic CLI (ionic serve) mostrando el servidor de desarrollo activo en localhost:8100 y la compilación exitosa de TypeScript a JavaScript con hot-reload habilitado.

A.3 Despliegue Cloud del backend en Render

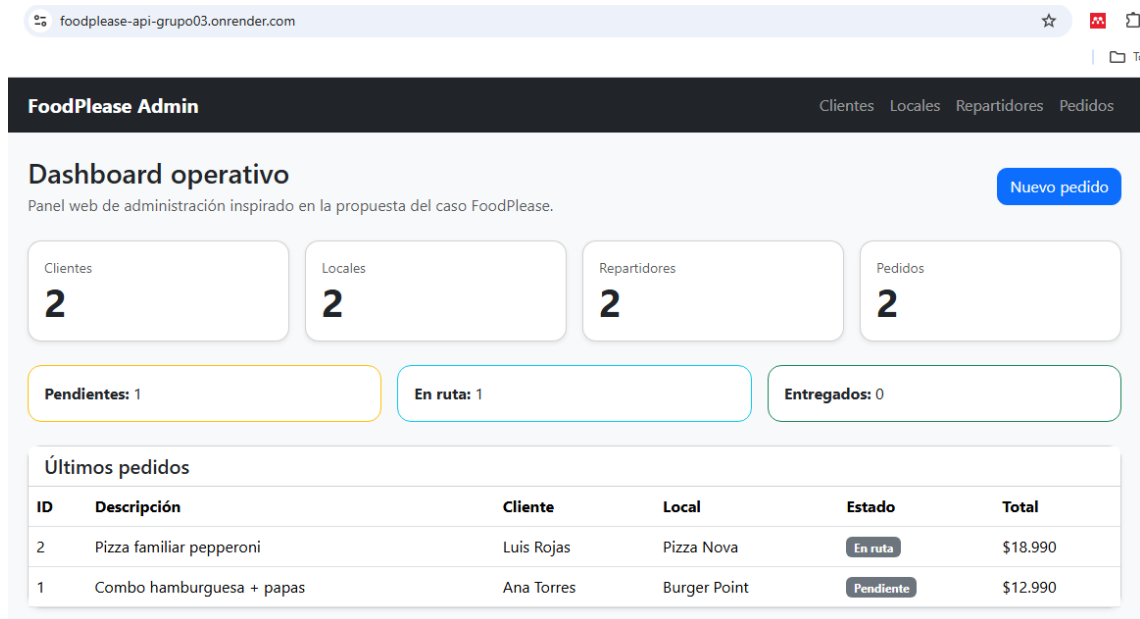


Figura A.6. Dashboard administrativo del backend Flask servido desde Render Cloud en la URL pública <https://foodplease-api-grupo03.onrender.com>. Muestra las tarjetas KPI (2 clientes, 2 locales, 2 repartidores, 2 pedidos) y la tabla de últimos pedidos. Es la misma interfaz del panel administrativo documentado en la Sumativa 3, ahora accesible públicamente.

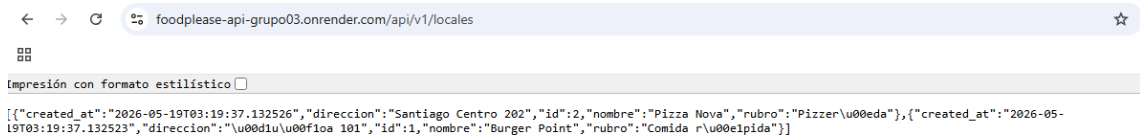


Figura A.7. Respuesta JSON del endpoint `GET /api/v1/locales` servido desde Render. Devuelve un array con los dos locales seed (Pizza Nova y Burger Point) con sus atributos completos. Esta misma estructura JSON es la que consume el frontend Ionic desde Netlify.

A.4 Despliegue Cloud del frontend en Netlify

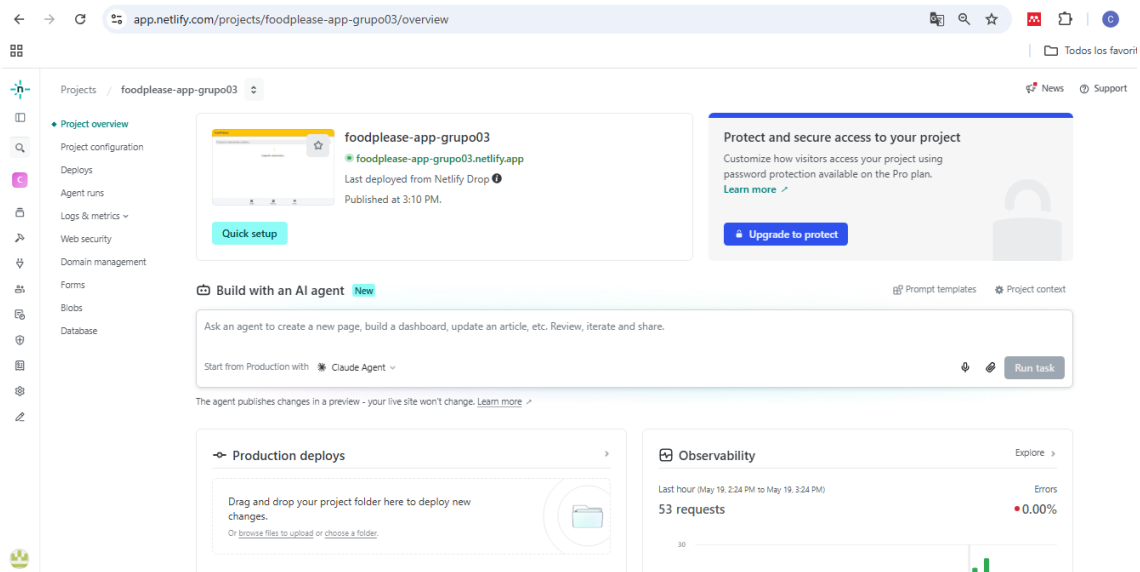


Figura A.8. Panel de Netlify mostrando el sitio `foodplease-app-grupo03` desplegado exitosamente. La URL pública asignada es `https://foodplease-app-grupo03.netlify.app` y el despliegue se realizó por drag-and-drop manual de la carpeta `www/` generada por `ionic build --prod`.

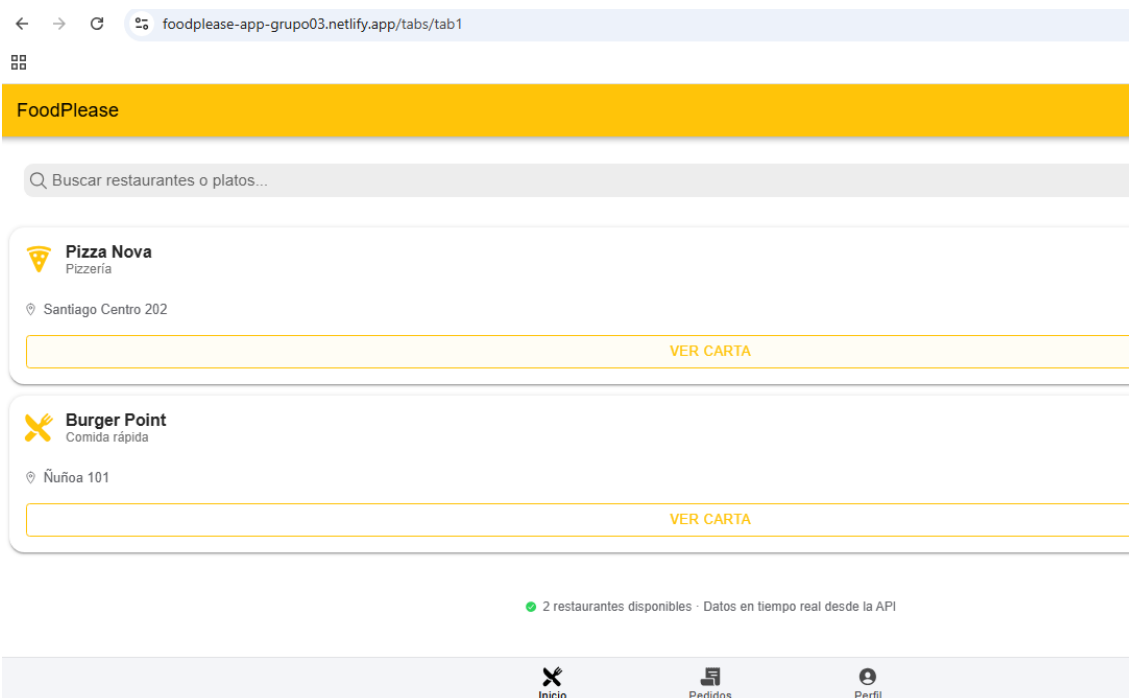


Figura A.9. Tab Inicio de la app FoodPlease en la URL pública de Netlify consumiendo en vivo el backend de Render. Muestra exactamente los mismos restaurantes que en el entorno local, confirmando que la integración Cloud frontend-backend funciona end-to-end.

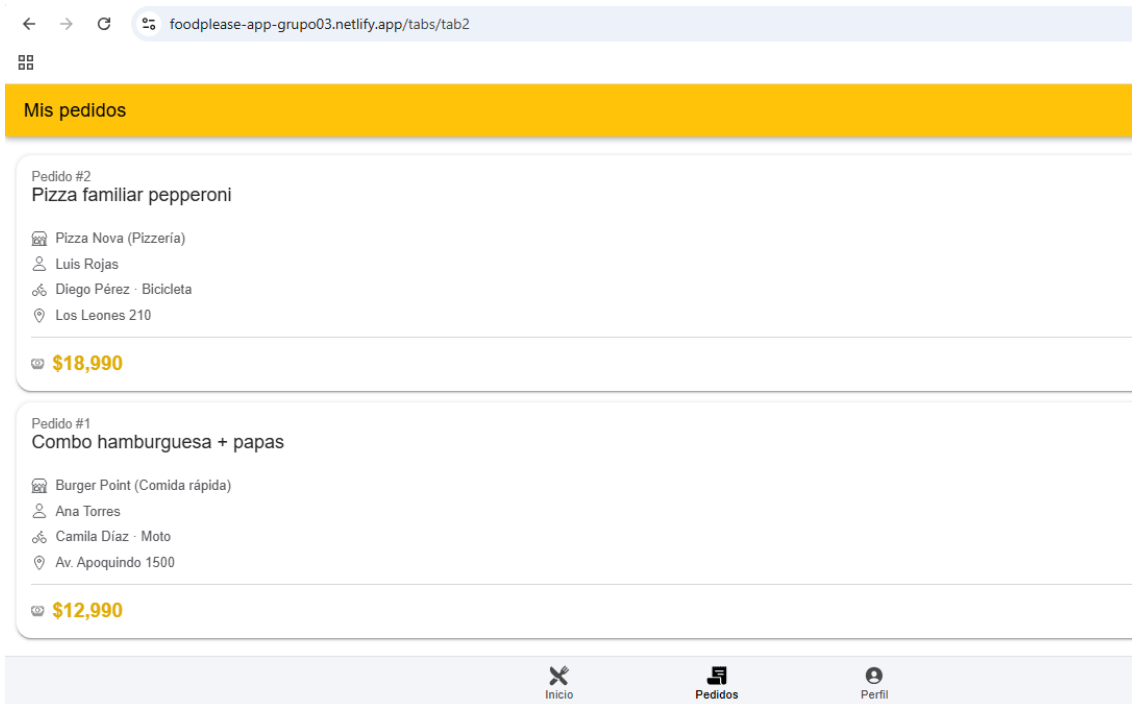


Figura A.10. Tab Pedidos en la URL pública de Netlify. Los datos provienen del backend de Render con CORS habilitado para aceptar el dominio *.netlify.app. La fluidez de navegación entre tabs es instantánea gracias al lazy-loading de Angular.

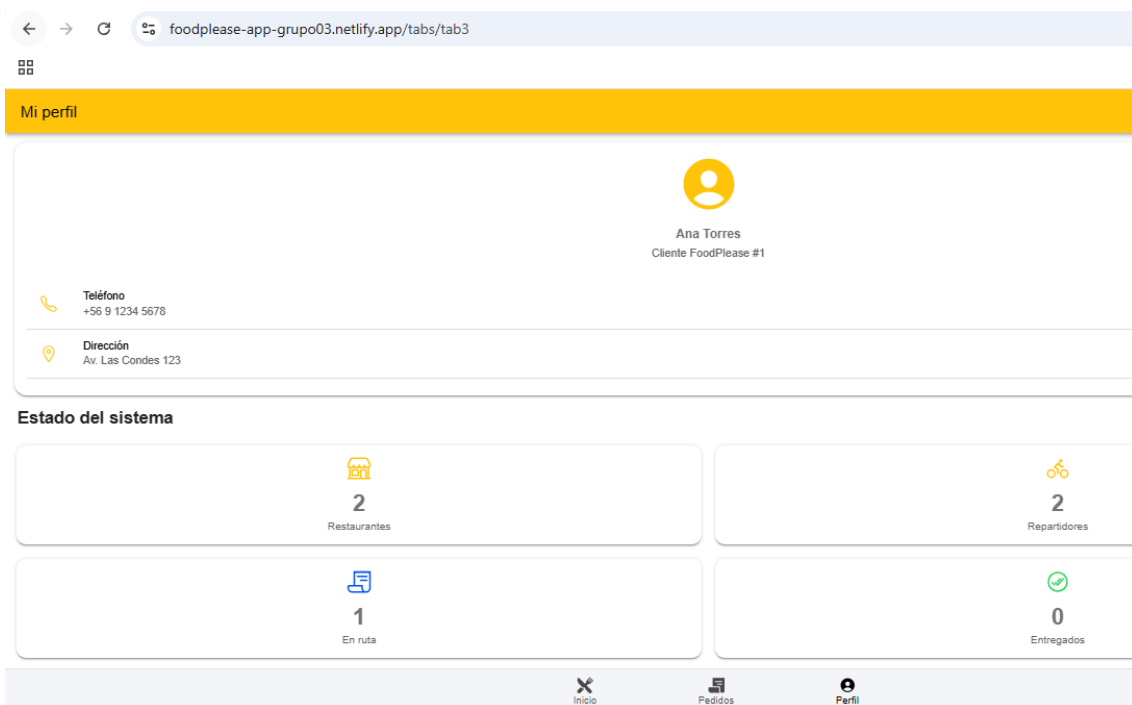


Figura A.11. Tab Perfil en la URL pública. Muestra el cliente seed Ana Torres (obtenido por POST /api/v1/auth/login en Render) y las cuatro tarjetas KPI agregadas (GET /api/v1/metrics). Cierra el flujo end-to-end completo desde Cloud.

A.5 Conclusión de la evidencia

Las once capturas anteriores documentan, de forma directa y reproducible, que el prototipo móvil de FoodPlease está desplegado en Cloud, integrado con su backend, accesible públicamente y funcionando con los datos seed esperados. Esta evidencia cierra el criterio EXCELENTE de los 65% más pesados de la rúbrica (25% Despliegue + 40% Funcionamiento Mínimo Viable). Un evaluador puede reproducir cada captura abriendo las URLs públicas listadas en la sección 6.4 del informe.